Reviewer Pack — Gröbner Basis Monomial Count and SAT Complexity

Entry 8315424c584f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 16:36:12 UTC

Gröbner Basis Monomial Count and SAT Complexity

Entry ID: `8315424c584f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 16:36:12 UTC

1. Conjecture

Field A (mathematical branch): Computational Algebraic Geometry **Field B** (complexity object): SAT Problem

Statement:

> For any unsatisfiable CNF formula with n variables, the number of monomials in the reduced Gröbner basis of its corresponding ideal over GF(2) is at least 2^{n/2}.

Rationale (proposer's reasoning):

> Gröbner bases capture algebraic dependencies of polynomial systems derived from CNF clauses. The monomial count reflects the complexity of resolving logical contradictions, potentially linking algebraic structure to circuit size lower bounds.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5f0213aa4bb3e35c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=1.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
        if 0 not in clause:
            clauses.append(clause)

    def poly_add(poly1, poly2):
        result = {}
        for term, coeff in poly1.items():
            result[term] = result.get(term, 0) + coeff
        for term, coeff in poly2.items():
            result[term] = result.get(term, 0) + coeff
        return {term: coeff % 2 for term, coeff in result.items() if coeff % 2 != 0}

    def poly_mul(poly1, poly2):
        result = {}
        for t1, c1 in poly1.items():
            for t2, c2 in poly2.items():
                new_term = tuple(sorted(t1 + t2))
                result[new_term] = result.get(new_term, 0) + c1 * c2
        return {term: coeff % 2 for term, coeff in result.items() if coeff % 2 != 0}

    def monomial_count(poly):
        return len(poly)

    def buchberger(clauses):
        G = [set([tuple(sorted(c))] for c in clauses)]
        while True:
            new_gens = set()
            for g1, g2 in itertools.combinations(G, 2):
                s = tuple(sorted(g1 | g2))
                if len(s) > len(g1) + len(g2):
                    h = poly_add(poly_mul(g1, {s: -1}), poly_mul(g2, {s: 1}))
                    for term in sorted(h.keys()):
                        if term[0] == s[-1]:
                            new_gens.add(term)
            if not new_gens:
                break
            G |= new_gens
```

```
        return monomial_count({tuple(sorted(g)): 1 for g in G})

    metric_value = buchberger(clauses)
    conjecture_holds = metric_value >= 2 ** (n // 2)
    counterexample = "" if conjecture_holds else f"n={n}, monomials={metric_value}"
    return {
        "metric_name": "monomial_count",
        "metric_value": metric_value,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 307))  # Generate 30 primes if no seed
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={results[0]['metric_name']}, monomials={resul
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26c9f5cf.py", line 78, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26c9f5cf.py", line 62, in run_trial
    metric_value = buchberger(clauses)
                   ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_26c9f5cf.py", line 47, in buchberger
    G = [set([tuple(sorted(c))] for c in clauses)]
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unhashable type: 'list'
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to unhashable type error, preventing metric evaluation | next: Fix the test code to handle clause lists properly and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106922
2	preregistration	ollama_remote	qwen3:8b	0	0	27467
3	novelty	ollama_remote	qwen3:8b	0	0	24052
4	novelty	ollama_remote	qwen3:8b	0	0	19132
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16126
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11723
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10581
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11464
9	judge	ollama_remote	qwen3:8b	0	0	111290

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 338757 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8315424c584f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8315424c584f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8315424c584f.tar.gz` (if generated)